

AI Assisted Coding

Name : M.Pravalika

H.no : 2303A52347

Batch : 34

Course Code : 23CS002PC304

Assignment Number : 15.5

Lab 15 – Backend API Development: Creating RESTful services with AI

Task 1 – Student Attendance API

Task:

Create a RESTful API for student attendance tracking.

Instructions:

- Endpoints required:
 - o GET /attendance → View attendance records
 - o POST /attendance → Mark attendance
 - o PUT /attendance/{id} → Update attendance record
 - o DELETE /attendance/{id} → Remove attendance entry
- Ensure JSON-based request and response handling.

Expected Output:

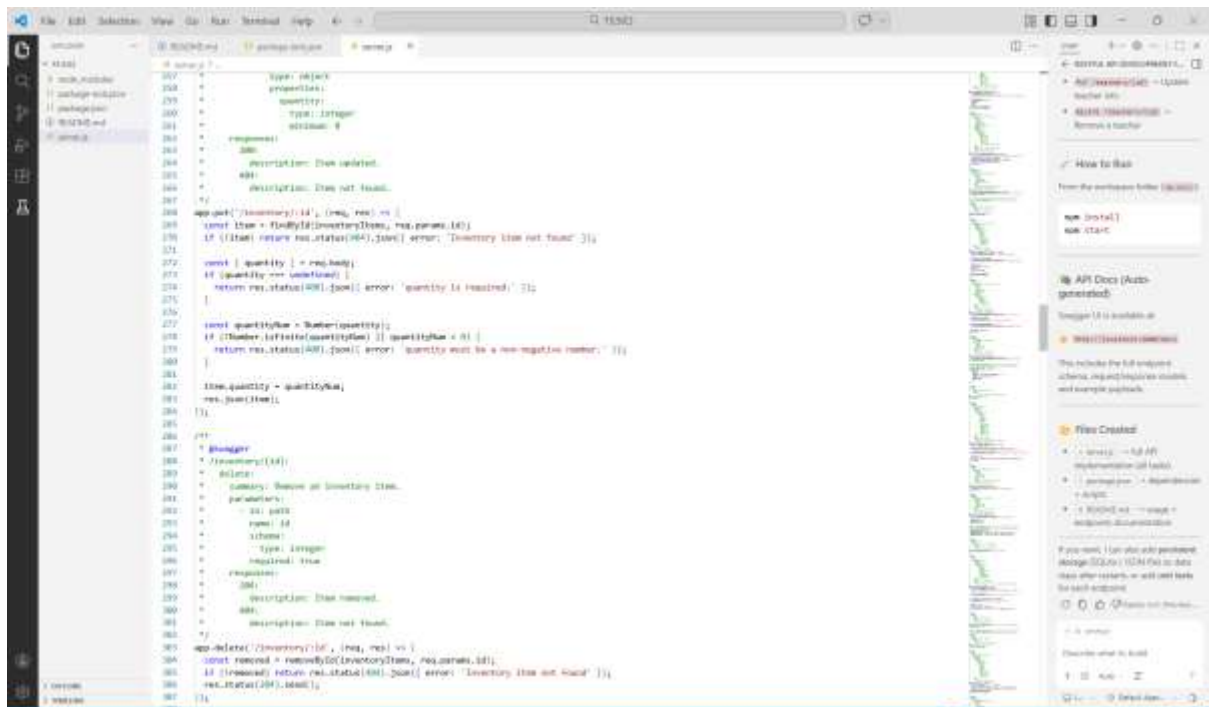
- API for maintaining and updating attendance records.

- Endpoints required:
 - o GET /inventory → List all items
 - o POST /inventory → Add new item
 - o PUT /inventory/{id} → Update stock quantity
 - o DELETE /inventory/{id} → Remove an item
 - Implement validation for stock values.
- Expected Output:
- API capable of managing inventory stock levels.

```

176 // Task 1 - Inventory Management API
177 //
178 //
179 /**
180  * @swagger
181  * /inventory:
182  *   get:
183  *     summary: List all inventory items.
184  *     responses:
185  *       200:
186  *         description: Inventory list.
187  */
188 app.get('/inventory', (req, res) => {
189   res.json(inventoryItems);
190 })
191
192 /**
193  * @swagger
194  * /inventory:
195  *   post:
196  *     summary: Add a new inventory item.
197  *     requestBody:
198  *       required: true
199  *     responses:
200  *       201:
201  *         description: Item added.
202  */
203 app.post('/inventory', (req, res) => {
204   const { name, description, quantity } = req.body;
205   if (!name || !quantity || quantity <= 0) {
206     return res.status(400).json({ error: 'name and quantity are required.' });
207   }
208   const newItem = {
209     name,
210     description,
211     quantity
212   };
213   inventoryItems.push(newItem);
214   res.status(201).json(newItem);
215 });
216
217 /**
218  * @swagger
219  * /inventory/{id}:
220  *   put:
221  *     summary: Update stock quantity.
222  *     parameters:
223  *       - name: id
224  *         in: path
225  *         required: true
226  *     requestBody:
227  *       required: true
228  *     responses:
229  *       200:
230  *         description: Stock updated.
231  */
232 app.put('/inventory/:id', (req, res) => {
233   const { id } = req.params;
234   const { quantity } = req.body;
235   if (!quantity || quantity <= 0) {
236     return res.status(400).json({ error: 'quantity must be a non-negative number.' });
237   }
238   const item = inventoryItems.find(item => item.id === id);
239   if (!item) {
240     return res.status(404).json({ error: 'Item not found.' });
241   }
242   item.quantity = quantity;
243   res.json(item);
244 });
245
246 /**
247  * @swagger
248  * /inventory/{id}:
249  *   delete:
250  *     summary: Remove an item.
251  *     parameters:
252  *       - name: id
253  *         in: path
254  *         required: true
255  *     responses:
256  *       200:
257  *         description: Item removed.
258  */
259 app.delete('/inventory/:id', (req, res) => {
260   const { id } = req.params;
261   const item = inventoryItems.find(item => item.id === id);
262   if (!item) {
263     return res.status(404).json({ error: 'Item not found.' });
264   }
265   inventoryItems = inventoryItems.filter(item => item.id !== id);
266   res.json({ message: 'Item removed.' });
267 });

```



Task 3 – Hotel Room Booking API

Task:

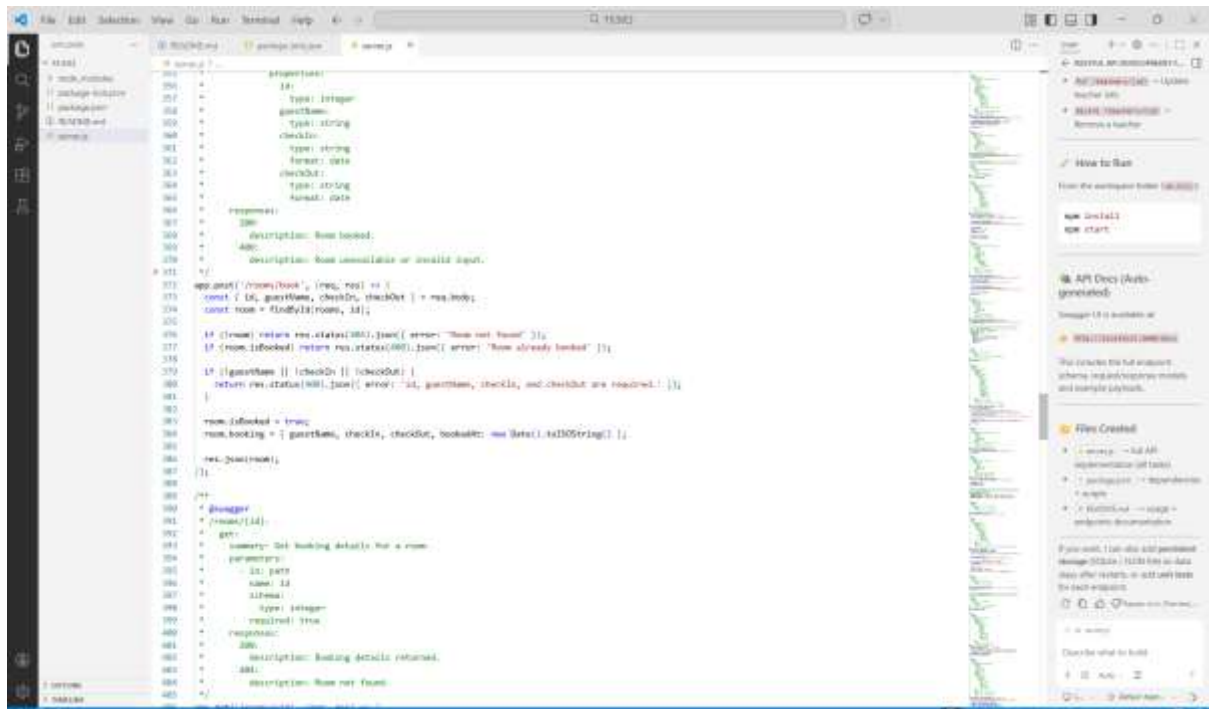
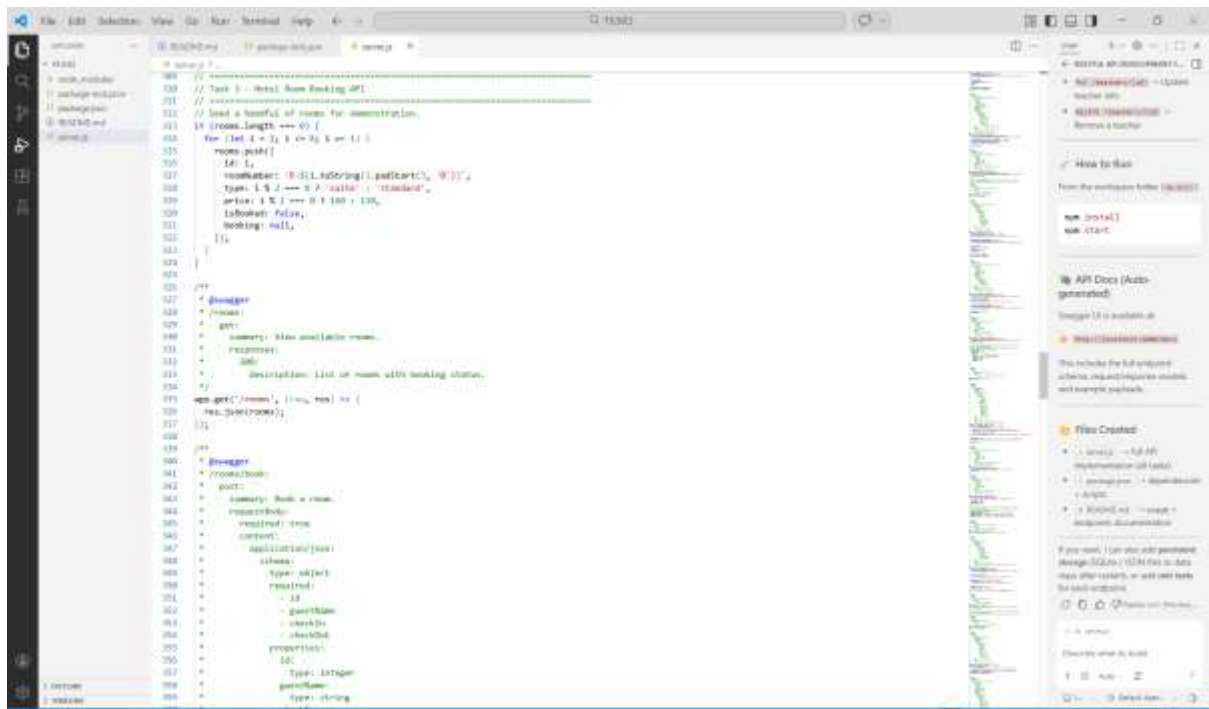
Develop a RESTful API for hotel room booking.

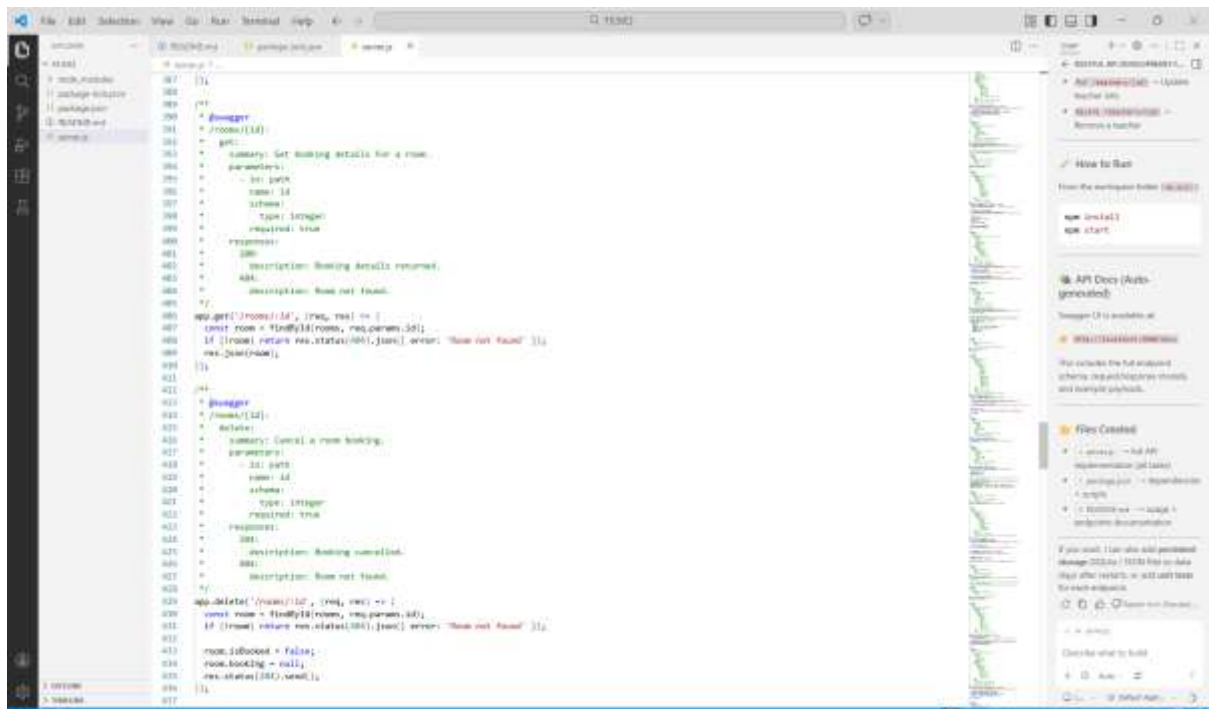
Instructions:

- Endpoints required:
 - o GET /rooms → View available rooms
 - o POST /rooms/book → Book a room
 - o GET /rooms/{id} → View booking details
 - o DELETE /rooms/{id} → Cancel booking
- Implement error handling for unavailable rooms.

Expected Output:

- API simulating hotel room booking operations.





Task 4 – Online Voting API

Task:

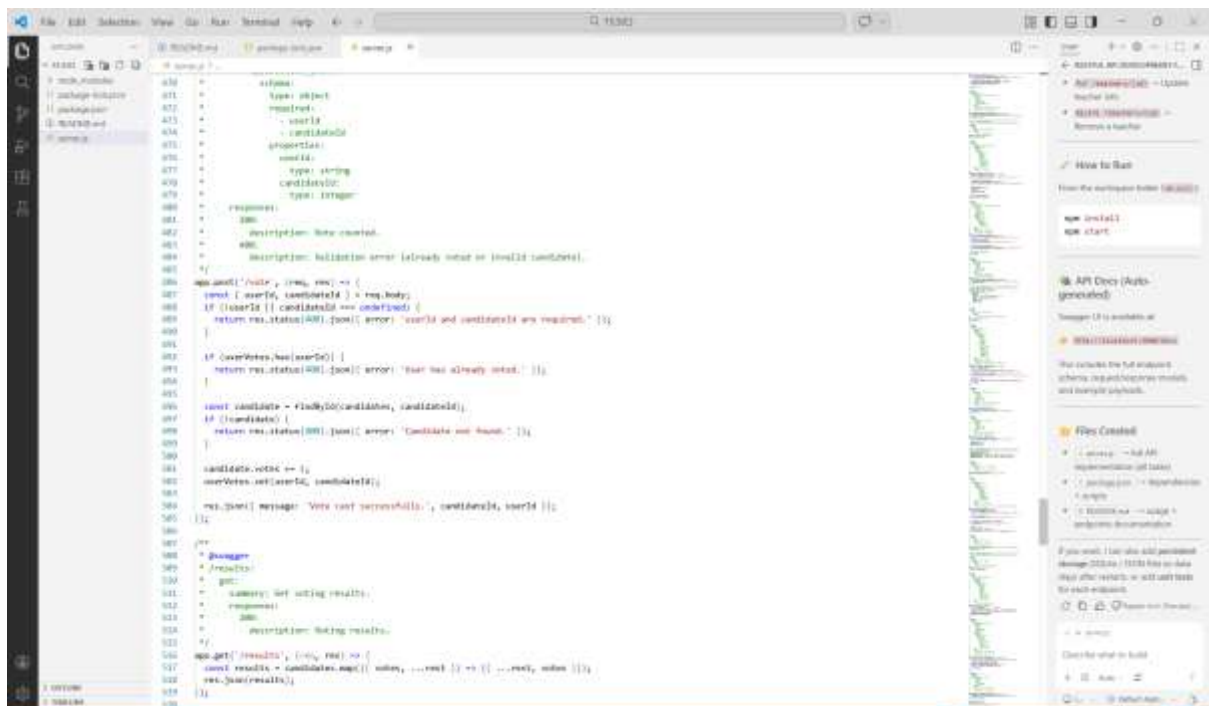
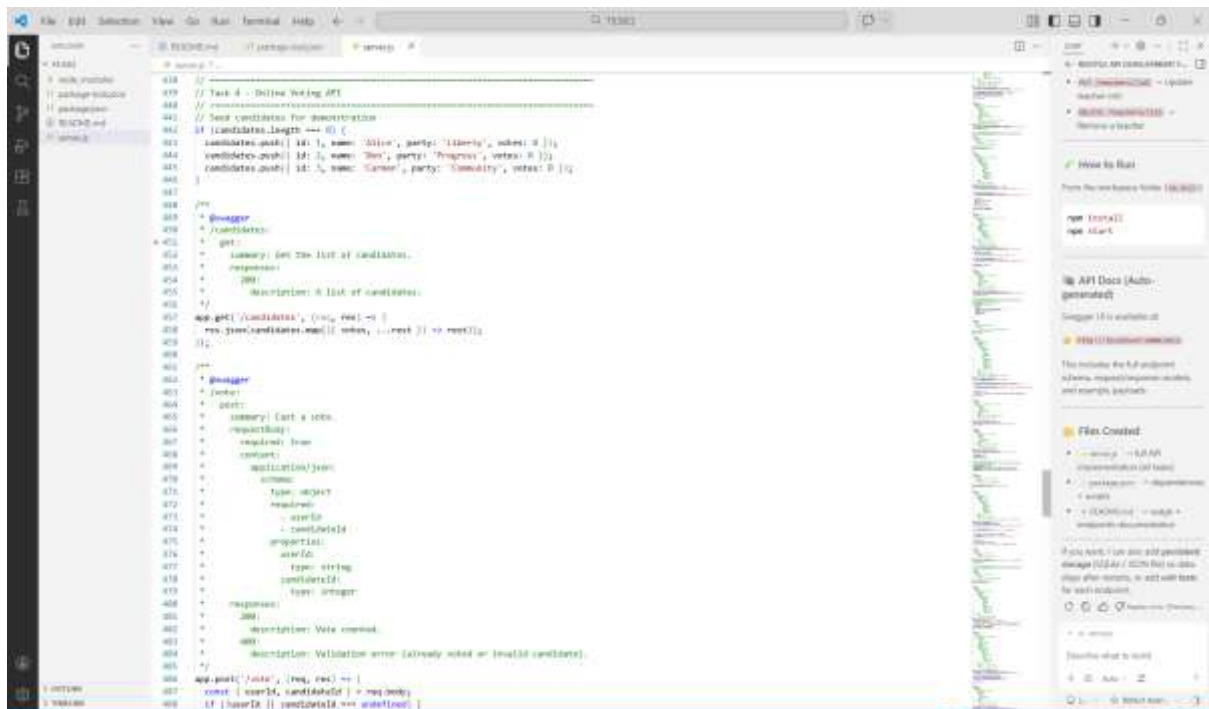
Create a RESTful API for an online voting system.

Instructions:

- Endpoints required:
 - o GET /candidates → List candidates
 - o POST /vote → Cast a vote
 - o GET /results → View voting results
- Ensure one vote per user using in-memory validation.

Expected Output:

- API simulating a secure online voting process.



Task 5 – Teacher Management API

Task:

Develop a RESTful API to manage teacher records.

Instructions:

- Endpoints required:

- o GET /teachers → List all teachers
 - o POST /teachers → Add a new teacher
 - o PUT /teachers/{id} → Update teacher details
 - o DELETE /teachers/{id} → Remove a teacher
 - Ensure proper error handling and JSON responses.
- Expected Output:
- Functional API for managing teacher information.

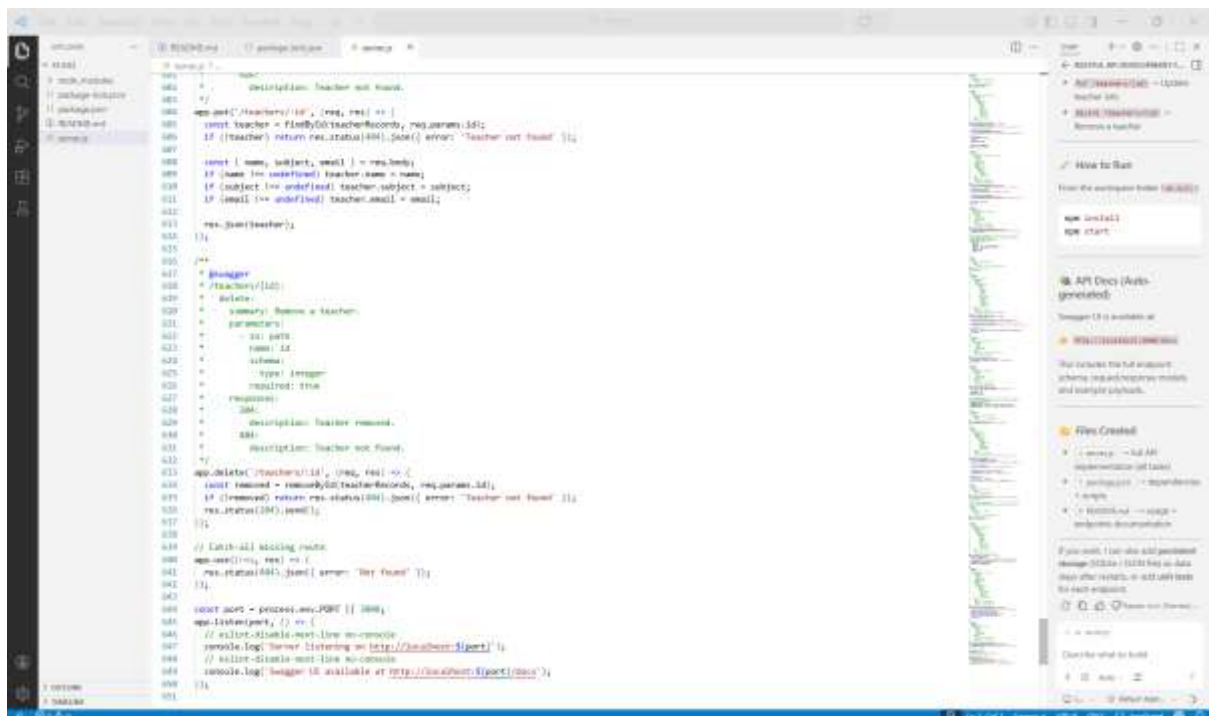
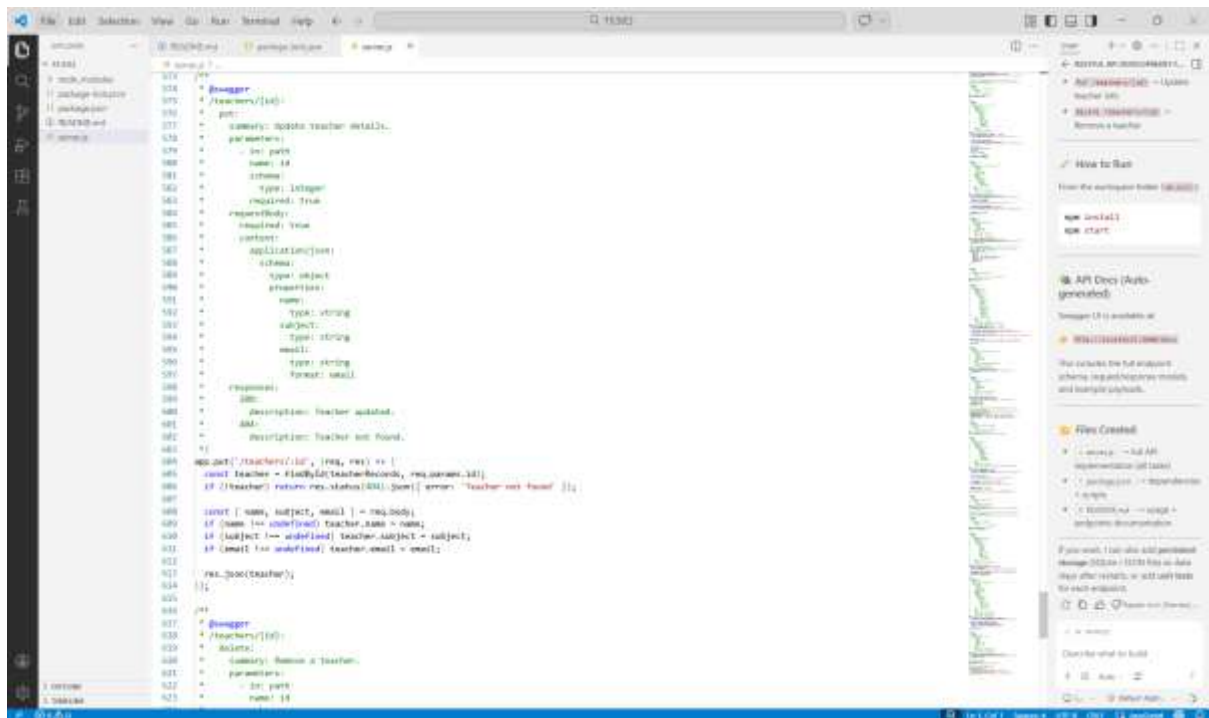
The screenshot shows a code editor with a REST API implementation for teacher management. The code is written in JavaScript using Express.js and includes Swagger documentation. The API endpoints are:

- GET /teachers: List all teachers. Returns a 200 status with a JSON array of teachers.
- POST /teachers: Add a new teacher. Requires a request body with 'name' and 'subject' fields. Returns a 201 status with a JSON object of the created teacher.

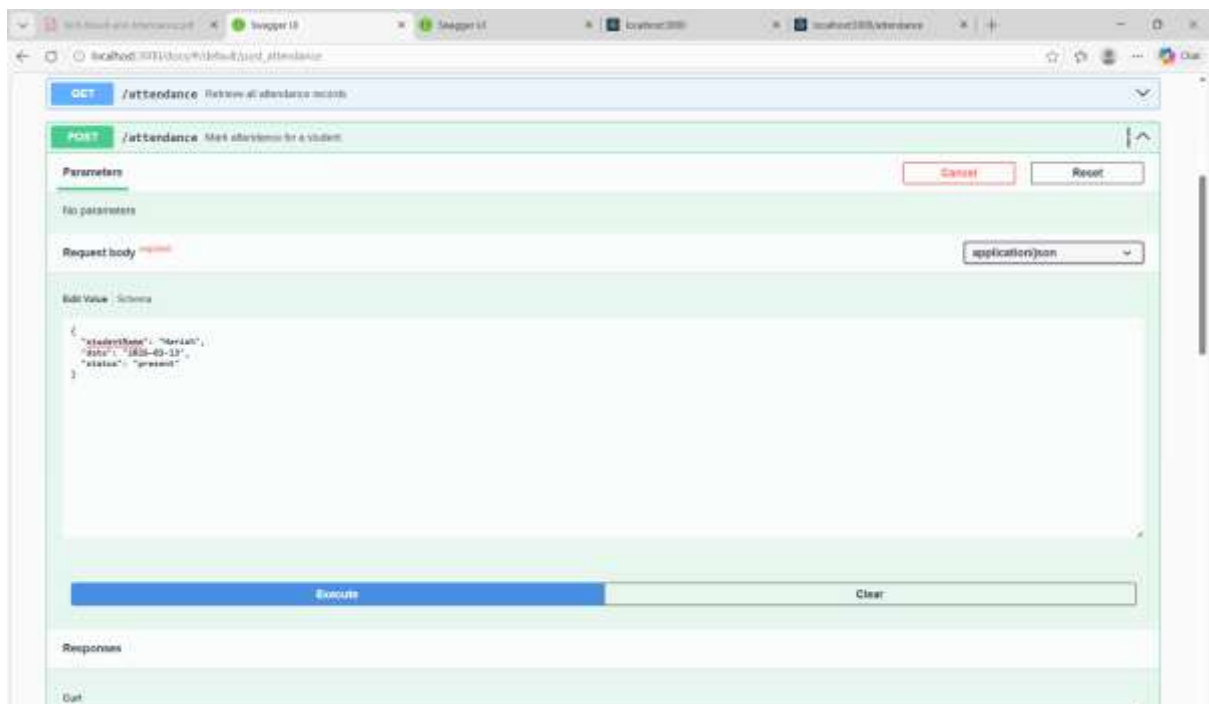
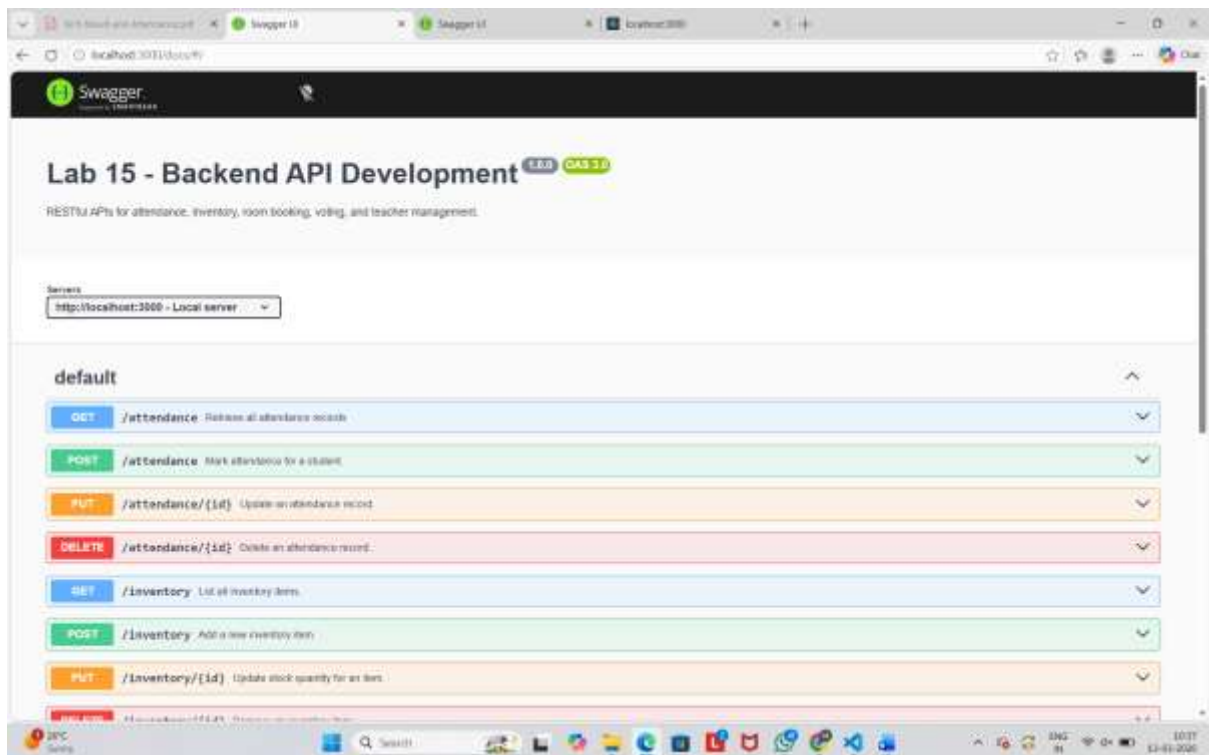
The code includes Swagger annotations for both endpoints, detailing the request and response formats. The GET endpoint returns a list of teachers, and the POST endpoint creates a new teacher record.

```

101 // GET /teachers
102 // Task 5 - Teacher Management API
103 //
104 // @summary List all teachers.
105 // @response 200 {
106 //   description: The list of teachers.
107 // }
108 app.get('/teachers', (req, res) => {
109   res.json(teacherRecords);
110 });
111
112 // POST /teachers
113 // @summary Add a new teacher.
114 // @requestBody {
115 //   required: true
116 //   schema: {
117 //     type: object
118 //     required: [
119 //       name
120 //       subject
121 //     ]
122 //     properties: {
123 //       name: {
124 //         type: string
125 //       }
126 //       subject: {
127 //         type: string
128 //       }
129 //     }
130 //   }
131 // }
132 // @response 201 {
133 //   description: Teacher created.
134 // }
135 app.post('/teachers', (req, res) => {
136   const { name, subject, email = '' } = req.body;
137   if (!name || !subject) {
138     return res.status(400).json({ error: 'name and subject are required.' });
139   }
140   const newTeacher = { id: teacherRecords.length + 1, name, subject, email };
141   teacherRecords.push(newTeacher);
142   res.status(201).json(newTeacher);
143 });
  
```

Backend API Development



Responses

Curl

```
curl -X 'POST' \
  -H 'Host: localhost:3000' \
  -H 'Content-Type: application/json' \
  -d '{
    "studentName": "Marish",
    "date": "2025-03-13",
    "status": "present"
  }'
```

Request URL

http://localhost:3000/attendance

Server response

Code

201

Response body

```
{
  "id": 1,
  "studentName": "Marish",
  "date": "2025-03-13",
  "status": "present"
}
```

Response headers

```
connection: keep-alive
content-length: 76
content-type: application/json; charset=utf-8
date: Fri, 13 Mar 2026 05:07:41 GMT
etag: W/"4d-wN21l88tCzjy8u2h0w4rky"
keep-alive: timeout=5
powered-by: Express
```

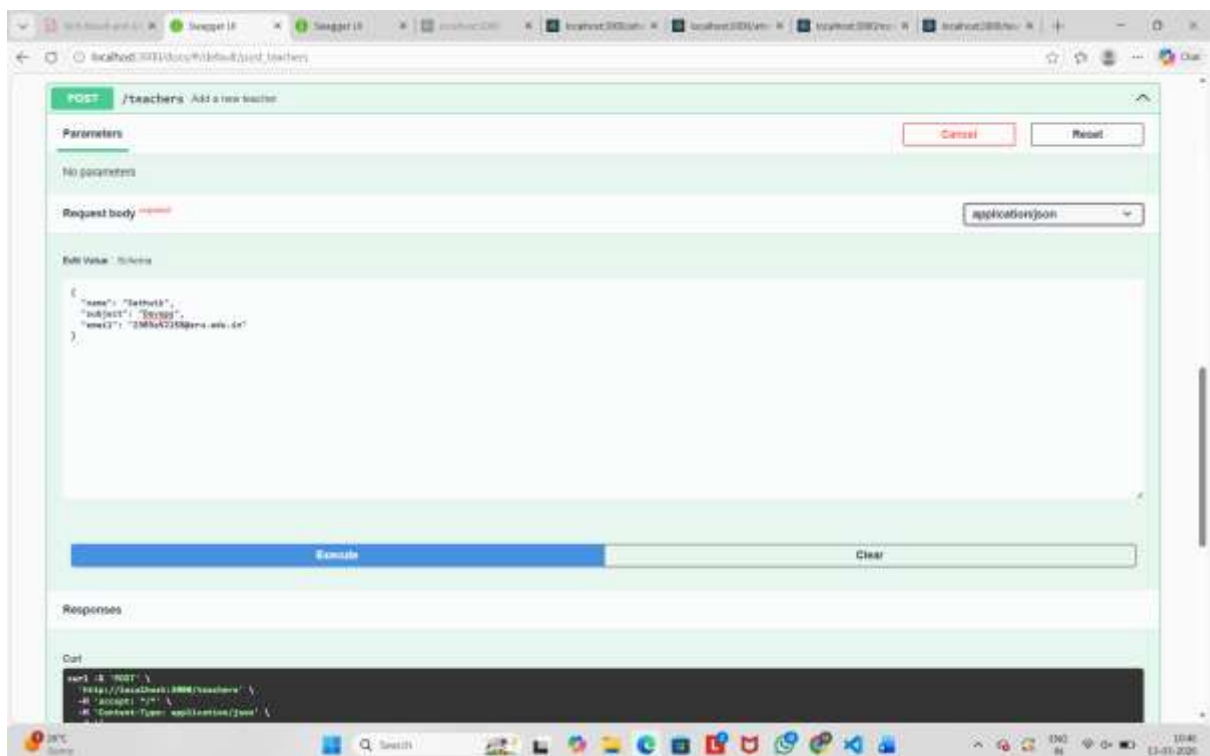
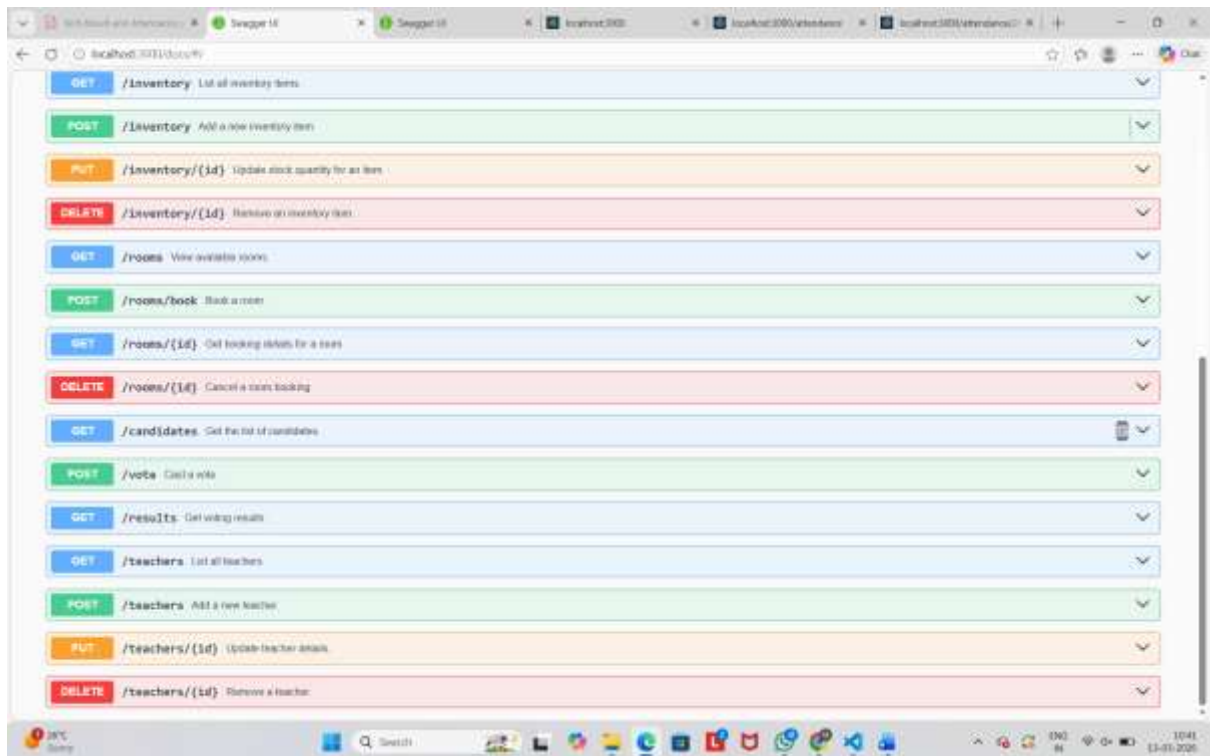
Responses

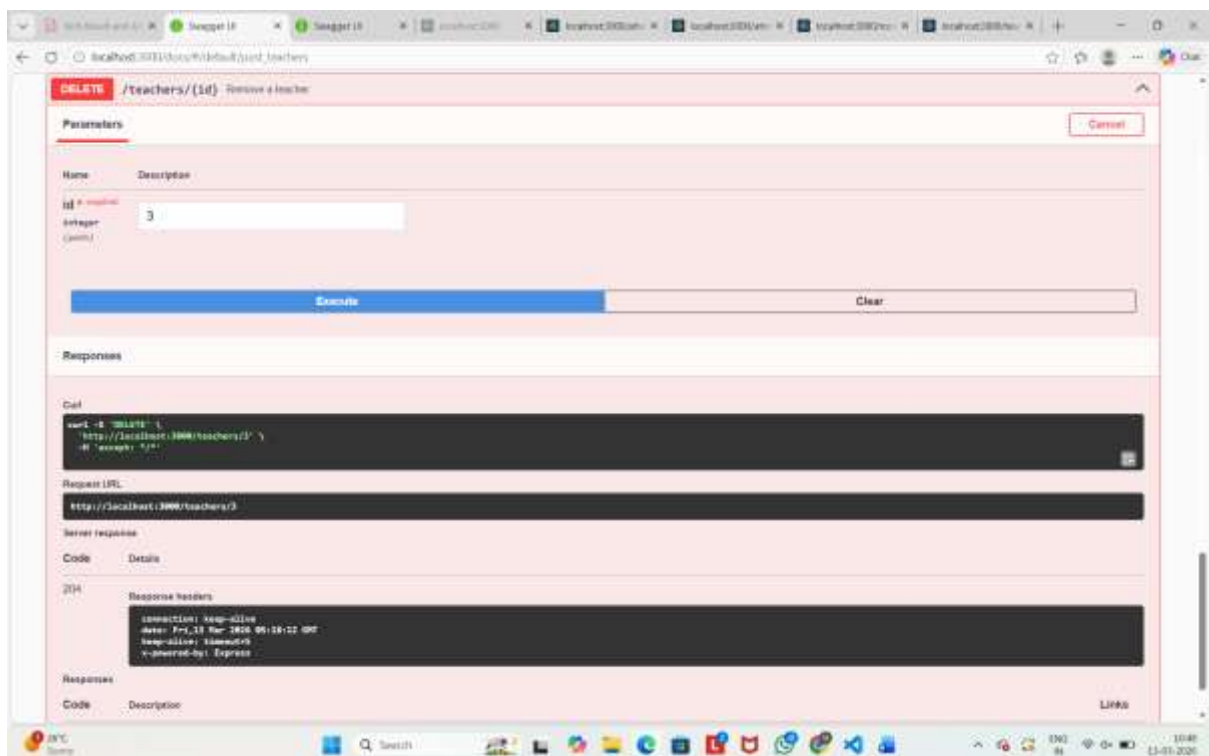
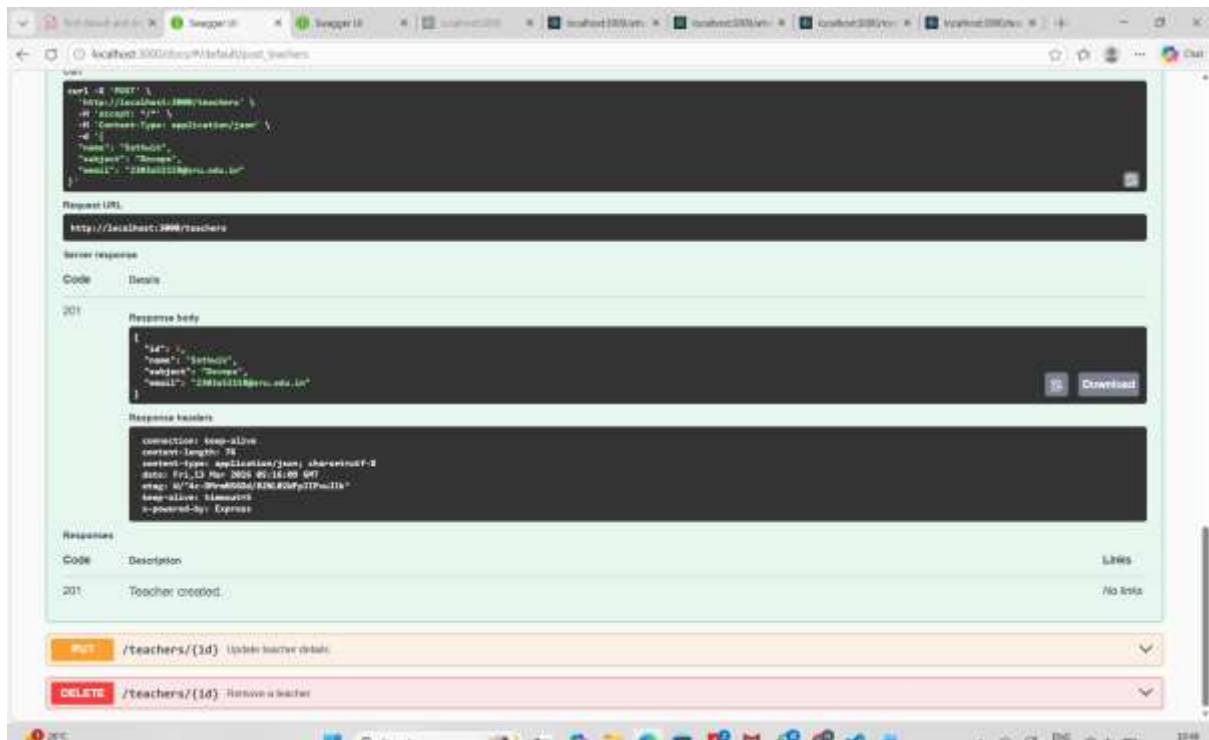
Code	Description	Links
201	Attendance recorded.	No links

POST /attendance/{id} Update an attendance record

Pretty-print

```
{
  "id": 1,
  "studentName": "Marish",
  "date": "2025-03-13",
  "status": "present"
},
{
  "id": 2,
  "studentName": "Marish",
  "date": "2025-03-13",
  "status": "present"
},
{
  "id": 3,
  "studentName": "Marish",
  "date": "2025-03-13",
  "status": "present"
}
```





GITHUB LINK : <https://github.com/2303A52387/AI-ASSISTANT-CODING/blob/main/15.zip>